UNIT – 2
NoSQL

Prepared by
Dr. K Samunnisa

Introduction to NoSQL

- **NoSQL** stands for **Not Only SQL**. It refers to a category of databases that are different from traditional relational databases.
- Traditional databases such as **MySQL, Oracle, SQL Server, PostgreSQL** store data in the form of **tables, rows, and columns**. These databases use SQL and are called **Relational Database Management Systems**, or **RDBMS**.

NoSQL databases are designed to handle:

- Very large data
- Fast data growth
- Flexible data formats
- Distributed storage
- High read and write speed
- Semi-structured and unstructured data

NoSQL databases are used when data is too large, too flexible, or too fast-changing for traditional relational databases.

Why NoSQL was introduced?

Earlier, most applications stored structured data.

For example: This type of data fits well in relational tables.

Roll No	Name	Branch	Marks
101	Ravi	CSE	85
102	Sita	DS	90

But modern applications generate different kinds of data:

- Facebook posts
- Instagram photos
- YouTube comments
- Amazon product reviews
- Mobile app logs
- Sensor data
- GPS location data
- Chat messages
- Clickstream data
- Shopping cart data

This data may not always fit neatly into fixed tables. That is why NoSQL databases became important.

Limitations of RDBMS in Big Data applications

RDBMS Limitation	Explanation
Fixed schema	Table structure must be defined in advance
Difficult horizontal scaling	Scaling across many servers is complex
Costly for huge data	Large relational systems need expensive servers
Less flexible for changing data	Adding new fields may require schema changes
Join operations are expensive	Large joins slow down performance
Not ideal for unstructured data	Images, logs, JSON, and social data are difficult to manage

Advantages of NoSQL

Advantage	Explanation
Flexible schema	Data structure can change easily
Horizontal scalability	Data can be distributed across many machines
High availability	System can continue even if one node fails
Fast read/write	Useful for real-time applications
Handles different data types	Suitable for text, JSON, logs, user activity, etc.
Suitable for Big Data	Designed for large-scale distributed systems

RDBMS vs NoSQL

Feature	RDBMS	NoSQL
Data model	Tables, rows, columns	Key-value, document, column-family, graph
Schema	Fixed schema	Flexible or schema-less
Query language	SQL	Varies by database
Scaling	Mostly vertical	Mostly horizontal
Data type	Structured data	Structured, semi-structured, unstructured
Joins	Supported	Usually avoided
Best use	Banking, ERP, academic records	Social media, IoT, e-commerce, analytics

Example:

- For a **college student marks database**, RDBMS is suitable because the data is structured.
- For an **e-commerce product catalogue**, NoSQL may be better because different products have different attributes.

Example:

A mobile phone has:

- RAM
- Camera
- Battery
- Processor

A book has:

- Author
- Publisher
- ISBN
- Edition

A shirt has:

- Size
- Color
- Fabric
- Sleeve type

All products do not have the same fields. So NoSQL is useful.

Aggregate Data Models

- A **data model** explains how data is organized, stored, and accessed in a database.
- In RDBMS, data is organized using tables and relationships.
- In NoSQL, many databases use **aggregate data models**.
- An **aggregate data model** stores related data together as one single unit.

In simple words:

- An aggregate is a collection of related data that is treated as one complete object.

Example: E-commerce order

Suppose a customer places an order on Amazon or Flipkart.

One order contains:

- Order ID
- Customer details
- Product details
- Quantity
- Price
- Delivery address
- Payment status
- Order date

In RDBMS, this may be stored in multiple tables:

Customer Table
Order Table
Order_Items Table
Product Table
Payment Table
Address Table

To display one order, we may need to join multiple tables.

Where as in NoSQL, we can store the full order as one aggregate.

JSON


```
{
  "order_id": "ORD101",
  "customer":
    {
    "customer_id": "C501",
    "name": "Ravi",
    "email": "ravi@gmail.com"
      },
  "items": [
    {
      "product_id": "P101",
      "product_name": "Laptop",
      "quantity": 1,
      "price": 55000
    },
     {
       "product_id": "P102",
       "product_name": "Mouse",
       "quantity": 1,
       "price": 700
     }
   ],
   "delivery_address": {
     "city": "Nandyal",
     "state": "Andhra Pradesh"
   },
   "payment_status": "Paid"
}
```


Here, the complete order is stored as one unit. This complete unit is called an **aggregate**.

Why aggregate data models are useful?

Aggregate models are useful because:

- Related data is stored together
- Data retrieval becomes faster
- Joins are reduced
- It is suitable for distributed databases
- It matches real-world objects
- It improves application performance

In RDBMS, we normally design data to avoid duplication using normalization.

In NoSQL, we often design data based on how the application reads and writes data.

So, in NoSQL:

- Data modeling is query-oriented, not only normalization-oriented.

Aggregates

An **aggregate** is a group of related data items that are handled as a single unit. It is the basic unit for:

- Storage
- Retrieval
- Update
- Consistency

Simple definition

- An **aggregate** is a complete data object that contains all related information required for a particular operation.

Example 1: Student aggregate: </>JSON

A student aggregate may contain:

```
{
  "student_id": 101,
  "name": "Sita",
  "branch": "CSE-DS",
  "semester": "IV-I",
  "subjects":
   [
    {
      "code": "A0517235",
      "name": "Big Data Analytics",
      "credits": 3
    },
    {
      "code": "A0525236",
      "name": "DevOps",
      "credits": 3
    }
  ]
}
```

Here, student details and subject details are stored together.

Example 2: Blog post aggregate

A blog post aggregate may contain:

```
{
  "post_id": 201,
  "title": "Introduction to Big Data",
  "author": "Dr. Kumar",
  "content": "Big Data is...",
  "comments":
    [
    {
      "user": "Ravi",
      "comment": "Good explanation"
    },
    {
      "user": "Anu",
      "comment": "Very useful"
    }
  ],
  "tags": ["Big Data", "Analytics", "NoSQL"]
}
```

The post, comments, author details, and tags are grouped together.

Aggregate design rule:

When designing an aggregate, ask:

“What data is usually accessed together?”

If data is commonly accessed together, it can be placed in the same aggregate.

Example

- For an online order page, order details, customer address, product list, and payment status are usually displayed together. So they can form one aggregate.
- But complete customer history may not be stored inside every order because it may become too large.

Advantages & Disadvantages of aggregates

Advantage	Explanation
Faster access	Related data is available in one place
Less joins	No need for many table joins
Better scalability	Easy to distribute aggregates across servers
Natural structure	Matches real-world business objects
Flexible design	Can store nested and complex data

Disadvantage	Explanation
Data duplication	Same data may be repeated
Update complexity	Updating repeated data can be difficult
Wrong aggregate design affects performance	Poor design may slow queries
Not suitable for all relationships	Complex many-to-many relationships may be difficult

Key-Value Data Model

A **key-value data model** stores data as a pair of:

Key → Value

The **key** is unique.
The **value** can be text, number, JSON, image, object, or any other data.

Simple definition

- **A key-value database** stores each data item using a unique key and its associated value.

Example:

101 → Ravi
102 → Sita
103 → Kumar

Another example:

user101 → {name: "Ravi", city: "Nandyal", branch: "CSE"}

Here, user101 is the key and the complete user details are the value.

Real-life comparison

A key-value database is like a dictionary.

For example:

Word → Meaning
Roll Number → Student Details
Mobile Number → Customer Details
Product ID → Product Details

You give the key, and the database returns the value.

Key-value data model diagram:

Key	**Value**
student101 →	Ravi, CSE, 85
student102 →	Sita, DS, 90
cart501 →	Laptop, Mouse, Keyboard
session999 →	Logged-in user details

Use cases of key-value databases

Key-value databases are useful for:

- Session storage
- Shopping cart
- User preferences
- Cache memory
- Login tokens
- Product lookup
- Configuration settings

Example: Shopping cart

cart_1001 → ["Laptop", "Mouse", "Keyboard"]

When the user logs in, the application uses cart_1001 to retrieve the cart items quickly.

Advantages of key-value databases

- Very fast
- Simple structure
- Easy to scale
- Good for lookup operations
- Suitable for caching and sessions

Limitations

- Not suitable for complex queries
- No joins
- Searching by value is difficult
- Relationships are hard to represent

Key-value databases are best when we know the key and want to quickly retrieve the related value.

Document Data Model

A **document data model** stores data in the form of documents.

These documents are usually represented using:

- JSON
- BSON
- XML

MongoDB commonly uses BSON, which is a binary form of JSON.

Simple definition

- **A document database** stores related data as flexible, self-contained documents.

Example of document data: </>JSON

```
{
  "student_id": 101,
  "name": "Ravi",
  "branch": "CSE-DS",
  "semester": "IV-I",
  "skills": ["Python", "SQL", "Machine Learning"],
  "address":
   {
    "city": "Nandyal",
    "state": "Andhra Pradesh"
   }
}
```

This single document contains all details about a student.

Important features of document databases

Feature	Explanation
Flexible schema	Each document can have different fields
Nested data	Documents can contain sub-documents
Array support	Documents can store lists
Easy for developers	Similar to objects in programming
Query support	Can search based on fields
Good for web apps	JSON format is common in APIs

Example: Different documents in same collection

Document 1: </>JSON

```
{
  "student_id": 101,
  "name": "Ravi",
  "email": "ravi@gmail.com"
}
```

Document 2: </>JSON

```
{
  "student_id": 102,
  "name": "Sita",
  "email": "sita@gmail.com",
  "phone": "9876543210",
  "skills": ["Java", "Python"]
}
```

Both documents can exist in the same collection even though the second document has extra fields. This is called **flexible schema**.

RDBMS table vs document database

- **RDBMS**

Student Table

---

student_id | name | email | phone | city

- **Document database**

```
{
  "student_id": 101,
  "name": "Ravi",
  "email": "ravi@gmail.com",
  "skills": ["Python", "SQL"],
  "address": {
    "city": "Nandyal"
     }
}
```

The structure can be flexible and nested.

Use cases of document databases

Document databases are useful for:

- User profiles
- Product catalogues
- Content management systems
- Blogs
- E-commerce applications
- Student records with flexible fields
- Customer records
- Mobile app data

Popular document databases

Database	Description
MongoDB	Popular JSON/BSON document database
CouchDB	Document database using JSON
Firebase Firestore	Cloud-based document database

Advantages of document databases

- Flexible schema
- Easy to store complex data
- Good for JSON-based applications
- Reduces joins
- Good for rapidly changing applications
- Better match with object-oriented programming

Limitations

- Data duplication may occur
- Complex transactions can be difficult
- Not always suitable for highly relational data
- Poor design may lead to large documents

Document databases are suitable when data is naturally represented as objects or JSON documents.

Key-Value vs Document Data Model

Feature	Key-Value Model	Document Model
Storage format	Key and value	JSON/BSON/XML document
Structure	Very simple	More structured than key-value
Querying	Mostly by key	By fields inside document
Flexibility	High	High
Best for	Cache, sessions, shopping cart	User profiles, product catalogues, blogs
Example	Redis	MongoDB
Relationship support	Very limited	Better than key-value, but still limited

Suppose we want to store student data.

Key-value model:

student101 → "Ravi, CSE-DS, 85"

This is simple and fast, but searching students by branch is difficult.

Document model:

```
{
    "student_id": 101,
    "name": "Ravi",
    "branch": "CSE-DS",
    "marks": 85
}
```

This is more flexible because we can search by branch, marks, or name.

Relationships in NoSQL Databases

- A **relationship** describes how one data item is connected to another data item.

Examples:

Customer → places → Order

Order → contains → Product

Student → enrolls in → Course

User → follows → User

- In a **relational database**, relationships are usually represented using **primary keys**, **foreign keys**, and **joins**.

Example:

Customer Table

Order Table

Product Table

Order_Items Table

- To get full order details, we may need to join many tables.

Relationships in aggregate-oriented NoSQL databases

In *NoSQL Distilled*, key-value, document, and column-family databases are called **aggregate-oriented databases**. These databases store related data together as an **aggregate**. However, relationships become difficult when the related data is present in different aggregates.

Example

Suppose an order document contains:

```JSON
{
  "order_id": 101,
  "customer_id": 501,
  "items": [
    {
      "product_id": 201,
      "product_name": "Laptop",
      "price": 55000
    }
  ]
}
```

Here, the order contains product information. But the customer_id connects this order to another customer aggregate.

Ways to handle relationships in NoSQL

1. Embedding

In embedding, related data is stored inside the same document or aggregate.

Example:

```json
{
  "customer_id": 501,
  "name": "Ravi",
  "orders": [
    {
      "order_id": 101,
      "product": "Laptop",
      "amount": 55000
    }
  ]
}
```

Advantages

- Faster reading
- No need for joins
- Good when data is usually accessed together

Disadvantages

- Document may become large
- Data duplication may occur
- Updating repeated data may become difficult

2. Referencing

In referencing, one aggregate stores the ID of another aggregate.

Example:

```json
{
  "order_id": 101,
  "customer_id": 501,
  "amount": 55000
}
```

Here, customer_id is used to connect the order with the customer document.

Advantages

- Reduces duplication
- Better when related data grows separately
- Useful when the related object is reused many times

Disadvantages

- Extra query may be required
- Application must manage the relationship
- NoSQL does not always support joins like RDBMS

3. Denormalization

Denormalization means storing duplicate data intentionally to improve read performance.

Example:

```json
{
  "order_id": 101,
  "customer_id": 501,
  "customer_name": "Ravi",
  "product": "Laptop",
  "amount": 55000
}
```

- Here, customer_name may already exist in the customer document, but it is repeated in the order document to avoid extra lookup.
- In RDBMS, we usually try to avoid duplication using normalization.
  In NoSQL, duplication is sometimes accepted to make reading faster.

Note:

- In NoSQL databases, relationships are not handled mainly through joins. Instead, NoSQL uses embedding, referencing, and denormalization.
- If related data is always accessed together, embedding is suitable.
- If related data grows independently, referencing is better.
- If read speed is more important, denormalization may be used.

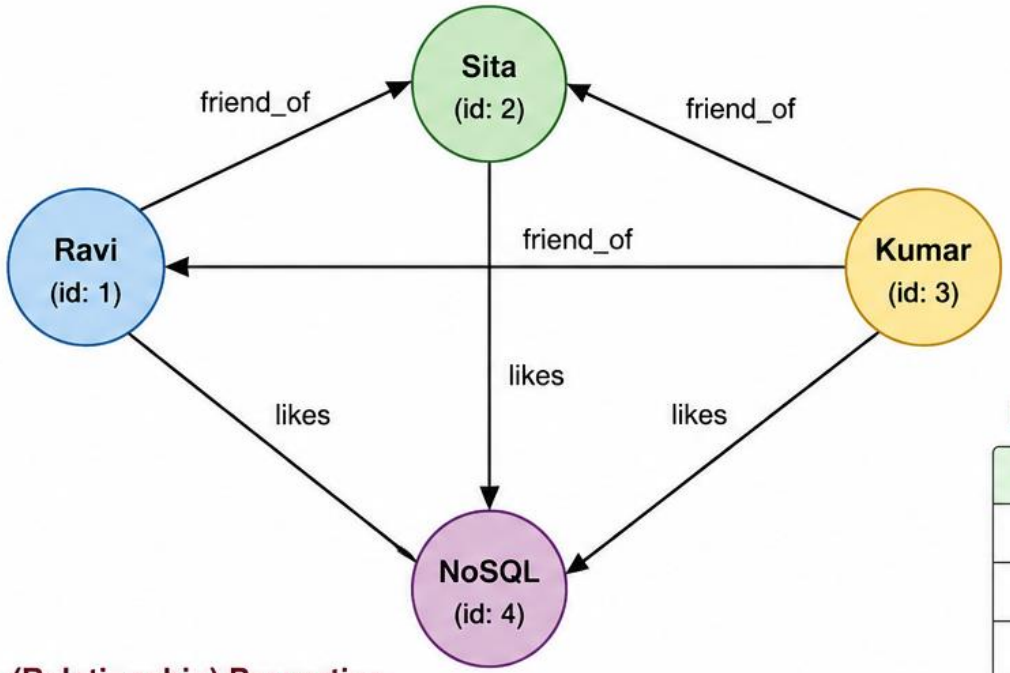
Graph Databases

- A **graph database** is a type of NoSQL database used to store and process data where relationships are very important.
- Graph databases store data using:

Component	Meaning	Example
Node	Entity or object	Student, User, Product, City
Edge	Relationship between nodes	follows, likes, bought, enrolled in
Property	Additional information	name, age, date, price

Graph Database Example – Social Network

Example: Ravi, Sita, and Kumar in a Social Network



Legend

- Person Node
- Topic Node
- Relationship (Edge)

Node Properties

Node (ID)	Label	Properties
1	Person	name: "Ravi", age: 25
2	Person	name: "Sita", age: 24
3	Person	name: "Kumar", age: 26
4	Topic	name: "NoSQL", type: "Technology"

Edge (Relationship) Properties

From	To	Relationship Type	Properties
1 (Ravi)	2 (Sita)	friend_of	since: "2022-01-10"
2 (Sita)	3 (Kumar)	friend_of	since: "2022-02-15"
3 (Kumar)	1 (Ravi)	friend_of	since: "2022-03-20"
1 (Ravi)	4 (NoSQL)	likes	since: "2023-05-05"
2 (Sita)	4 (NoSQL)	likes	since: "2023-06-10"
3 (Kumar)	4 (NoSQL)	likes	since: "2023-07-12"

Example Traversal Query

Question: Which friends of Ravi like NoSQL?

Traversal:

Ravi → friend_of → (Sita, Kumar)
→ likes → NoSQL

Result:

Sita, Kumar

Graph database vs relational database

- In relational databases, relationships are usually found by performing **joins**. But in graph databases, relationships are stored directly as **edges**. Because of this, traversing relationships becomes **faster** in highly connected data.
- Graph databases **reduce expensive join** operations because the relationships are already stored and can be traversed directly.

Feature	Relational Database	Graph Database
Basic structure	Tables, rows, columns	Nodes, edges, properties
Relationship handling	Foreign keys and joins	Direct edges
Best for	Structured transactional data	Highly connected data
Query style	SQL joins	Graph traversal
Example use	Student marks, banking records	Social networks, recommendations

Example: Recommendation system

Ravi → bought → Laptop

Ravi → bought → Mouse

Sita → bought → Laptop

Sita → bought → Mouse

Sita → bought → Laptop Bag

Since Sita bought similar products and also bought a laptop bag, the system may recommend **Laptop Bag** to **Ravi**.

Applications of graph databases

Application	Use
Social networks	Friend/follower relationships
Recommendation systems	Product, movie, or friend suggestions
Fraud detection	Detect hidden suspicious links
Route finding	Shortest path and navigation
Knowledge graphs	Connect concepts and entities
Cybersecurity	Attack path analysis
Supply chain	Supplier-product-customer relationships

Advantages of graph databases

- Best for connected data
- Relationship traversal is fast
- Handles many-to-many relationships naturally
- Easy to add new types of relationships
- Useful for recommendation and fraud detection
- Relationships can also have properties

Limitations of graph databases

- Not ideal for simple tabular data
- Scaling across many machines can be difficult
- Not always suitable for large batch analytics
- Requires graph-based thinking
- Query languages may be different from SQL

Schemaless Databases

- A **schema** is the structure of a database. In a relational database, before storing data, we must define tables, columns, data types, and constraints.
- Example:

```sql
CREATE TABLE Student (
  student_id INT,
  name VARCHAR(50),
  branch VARCHAR(20),
  marks INT
);
```

- Here, every student record must follow the same structure.

Where as in a **schemaless database,** it does not force a fixed structure before storing data.

In simple words:

- A schemaless database allows different records to have different fields.

Example

Document 1:

```json
{
  "student_id": 101,
  "name": "Ravi",
  "branch": "CSE-DS"
}
```

Document 2:

```json
{
  "student_id": 102,
  "name": "Sita",
  "branch": "CSE-DS",
  "email": "sita@gmail.com",
  "skills": ["Python", "Java"]
}
```

Both documents can be stored in the same collection even though the second document has extra fields.

Example: Product catalogue

A mobile phone may have:

```JSON
{
  "product_id": "P101",
  "name": "Smartphone",
  "ram": "8GB",
  "camera": "64MP",
  "battery": "5000mAh"
}
```

A shirt may have:

```JSON
{
  "product_id": "P102",
  "name": "Cotton Shirt",
  "size": "L",
  "color": "Blue",
  "fabric": "Cotton"
}
```

Both are products, but their attributes are different. A schemaless database can store both easily.

Note: Schemaless does **not** mean there is no structure at all. Even if the database does not enforce a schema, the application code usually expects certain field names and data meanings. This is called an **implicit schema**.

Example:

If the application expects:

```
{
  "quantity": 5
}
```

but another document stores:

```
{
  "qty": "five"
}
```

then the application may **fail or produce wrong output**.

Why schemaless databases are useful

Schemaless databases are useful when:

- Data structure changes frequently
- Different records have different attributes
- Application development is fast and evolving
- Data comes from many sources
- Semi-structured data such as JSON is used
- New fields need to be added without changing the whole database

Advantage of schemaless databases	Explanation
Flexibility	Easy to add new fields
Faster development	No frequent schema migration
Handles non-uniform data	Different records can have different fields
Good for JSON data	Suitable for web and mobile applications
Supports evolving requirements	Useful when design changes often

Limitation of schemaless databases	Explanation
Data inconsistency	Same meaning may be stored using different field names
Application dependency	Structure is controlled by application code
Validation difficulty	Wrong or missing fields may not be caught early
Poor documentation risk	Developers may not know the real structure
Query complexity	Irregular data may be hard to query

Materialized Views

- A **view** is the result of a query. In relational databases, a normal view may act like a virtual table.
- A **materialized view** is a precomputed query result that is stored physically.

In simple words:

- A materialized view stores the answer of a frequently used query so that the system can return the result faster next time.
- NoSQL databases may not have views in the relational sense, but they often use precomputed or cached query results called **materialized views**. These are very important in aggregate-oriented databases because some queries may not match the main aggregate structure.

Why materialized views are needed in NoSQL

- In NoSQL, data is often stored based on one access pattern. But applications may need different views of the same data.

Example: Original data may be stored order-wise:

```json
{
  "order_id": 101,
  "customer_id": 501,
  "city": "Nandyal",
  "amount": 5000
}
```

But the application may frequently ask:

What is the total sales amount city-wise?

If the system calculates this every time from millions of orders, it becomes slow.

So, we store a materialized view:

City Total Sales

Nandyal 5000

Hyderabad 250000

Bangalore 320000

Example: Product review summary

Suppose an e-commerce website stores product reviews.

Original reviews:

```json
{
  "product_id": "P101",
  "reviews": [
    {"rating": 5},
    {"rating": 4},
    {"rating": 3}
  ]
}
```

Frequently needed result:

Product ID: P101

Number of reviews: 3

Average rating: 4.0

- Instead of calculating count and average rating every time, the system can store this as a materialized view.

Two strategies for updating materialized views

1. Eager update

The materialized view is updated immediately when base data changes.

Example:

When a new order is added, city-wise sales summary is updated at the same time.

Advantage

- View is fresh and updated
- Good for frequently read reports

Disadvantage

- Write operation becomes slower
- Application must update both base data and view

2. Batch update

The materialized view is updated periodically.

Example:

City-wise sales summary is updated every hour or every night.

Advantage

- Faster write operations
- Suitable for large analytics

Disadvantage

- View may be slightly old or stale
- Not suitable when real-time accuracy is required

The textbook explains both approaches: one where the view is updated when base data changes, and another where batch jobs refresh the view periodically depending on how much stale data the business can tolerate.

Materialized views inside aggregates

- A materialized view can also exist inside the same aggregate.

Example:

```JSON
{
  "order_id": 101,
  "items": [
    {"product": "Laptop", "price": 55000},
    {"product": "Mouse", "price": 700}
  ],
  "order_summary": {
    "total_items": 2,
    "total_amount": 55700
  }
}
```

Here, order_summary is a precomputed result stored inside the order document.

Advantage of materialized views	Explanation
Faster reads	Query result is already stored
Less repeated computation	Same calculation need not be repeated
Useful for dashboards	Reports can load faster
Supports different access patterns	Same data can be organized in different ways
Helpful in NoSQL	Reduces need for joins and complex queries

Limitation of materialized views	Explanation
Extra storage	Same data/result is stored separately
Update complexity	View must be refreshed when base data changes
Stale data	View may not always show latest data
More write overhead	Eager update increases write cost
Data duplication	Same information exists in multiple forms

Note:

- A materialized view is a stored result of a query.
- It improves read performance by avoiding repeated computation.
- In NoSQL databases, materialized views are important because data is often stored according to one main aggregate structure, while applications may need different query formats.
- Materialized views can be updated immediately or through periodic batch processing.

Distribution Models in NoSQL

- A **distribution model** explains how a database stores and manages data across multiple machines or nodes.
- In traditional databases, data is usually stored on one powerful server.
- But in Big Data applications, data volume and traffic become very large. Therefore, instead of using one big server, NoSQL databases often use a **cluster of servers**. This is called **scaling out**.
- NoSQL became popular mainly because it can run databases on large clusters, and aggregate-oriented data fits well with distribution because an aggregate becomes a natural unit for distribution.

Why distribution is needed

Distribution is used to achieve:

- Large data storage
- Better read/write performance
- High availability
- Fault tolerance
- Load balancing
- Horizontal scalability

Two main distribution techniques

Technique	Meaning
Sharding	Different data is stored on different nodes
Replication	Same data is copied on multiple nodes

- Replication and sharding can be used separately or together.
- Replication has two forms: **master-slave replication** and **peer-to-peer replication**.

Sharding

- **Sharding** means dividing a large dataset into smaller parts and storing each part on a different server.
- Each smaller part is called a **shard**.

In simple words:

Sharding stores different portions of data on different machines.

- Sharding supports horizontal scalability by placing different parts of the data on different servers. In Figure 2.1, each node stores different data and handles its own reads and writes.

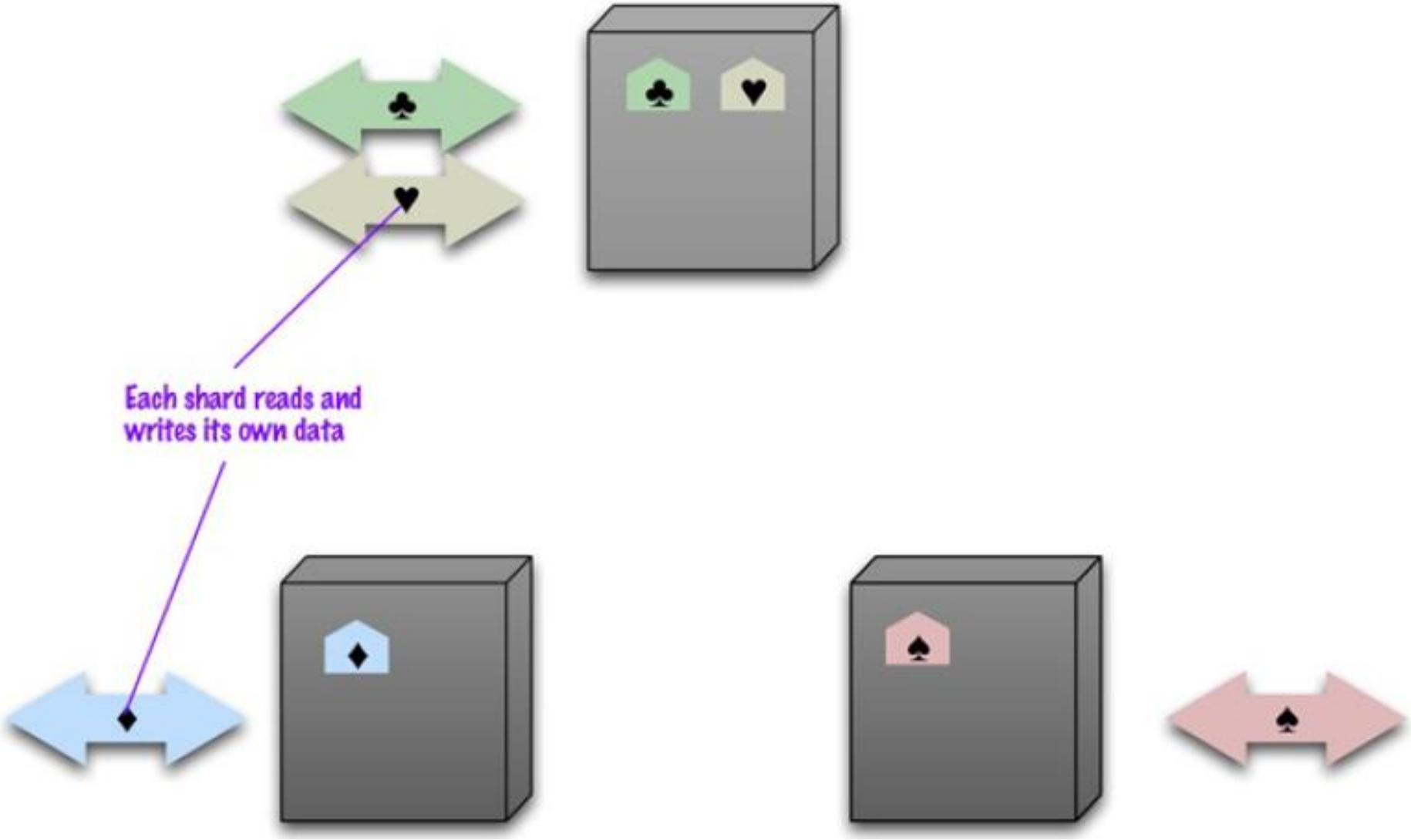


Figure 2.1. Sharding puts different data on separate nodes, each of which does its own reads and writes.

Example

Suppose we have student records from roll numbers 1 to 3000.

- Roll No 1–1000 → Server 1
- Roll No 1001–2000 → Server 2
- Roll No 2001–3000 → Server 3

- Each server stores only one part of the full data.

Advantages of sharding

- Handles very large data
- Improves scalability
- Distributes load across servers
- Reduces pressure on a single server
- Improves performance when users access different data portions

Challenges of sharding

- Choosing the correct shard key is difficult
- Data may become unevenly distributed
- Queries across multiple shards are complex
- Rebalancing data is difficult when new nodes are added
- Related data should be kept together as much as possible

Master-Slave Replication

- **Master-slave replication** means copying the same data from one main node to other nodes.
- The main node is called the **master** or **primary**. The copied nodes are called **slaves** or **secondaries**.
- The master is the authoritative source and usually handles updates, while slave nodes are synchronized with the master.
- In Figure 2.2, data is replicated from master to slaves; the master handles all writes, while reads may come from either master or slaves.

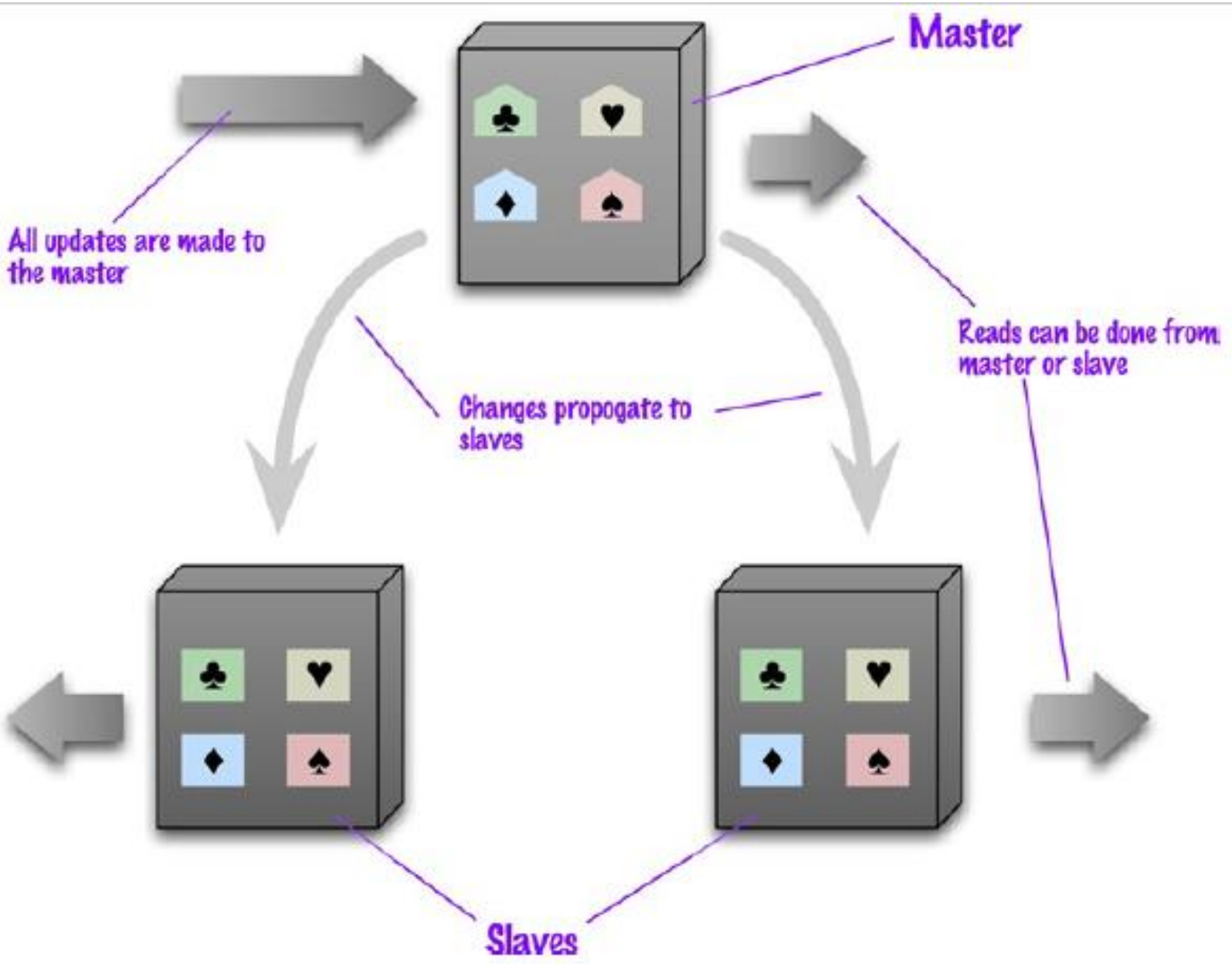


Figure 2.2: Data is replicated from master to slaves. The master services all writes; reads may come from either master or slaves.

How it works

1. Application sends write requests to the master.
2. Master updates the data.
3. Changes are copied to slave nodes.
4. Read requests can be served by slave nodes.
5. If the master fails, a slave may be promoted as new master.

Example

In an e-commerce website:

- Master stores latest order and payment updates.
- Slave nodes are used to show product details and order history to users.
- Since many users read data, slaves reduce the load on the master.

Advantages

- Good for read-heavy applications
- Improves read performance
- Provides backup copies
- Supports read resilience
- A slave can become master if the master fails

Limitations

- Master is still a single point for writes
- Not suitable for heavy write traffic
- Replication delay may occur
- Different slaves may show different values temporarily
- If master fails before copying updates, some updates may be lost

Peer-to-Peer Replication

- **Peer-to-peer replication** means all nodes are equal. There is no single master.
- Any node can accept read and write requests.

In simple words:

- In peer-to-peer replication, every node can read, write, and synchronize data with other nodes.

Peer-to-peer replication solves the master bottleneck problem because all replicas have equal weight and can accept writes. If one node fails, the system can still access the data through other nodes.

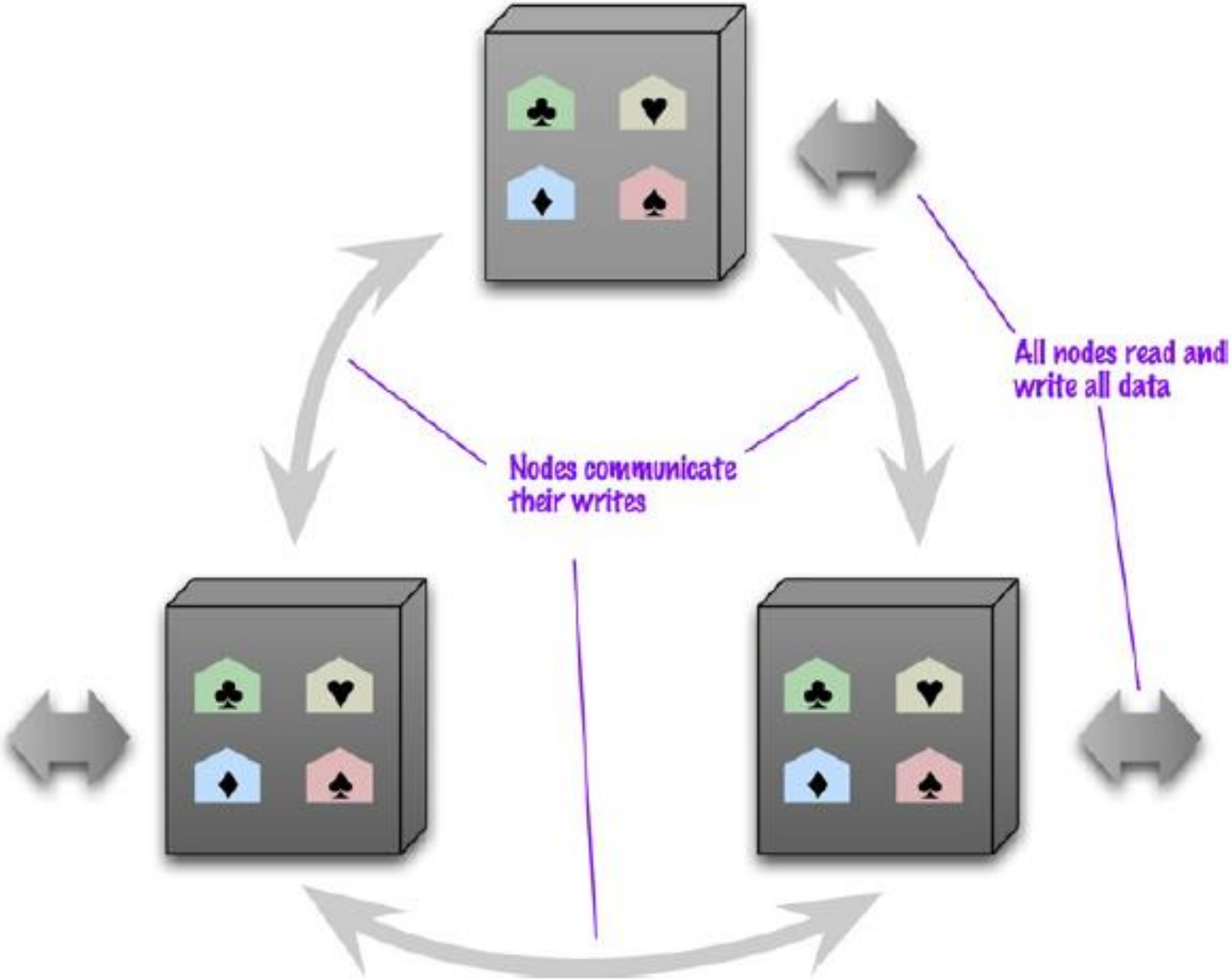


Figure 2.3. Peer-to-peer replication has all nodes applying reads and writes to all the data.

How it works

1. Data is copied on multiple nodes.
2. Any node can receive a write request.
3. Nodes communicate with each other to synchronize updates.
4. If one node fails, other nodes continue working.
5. When the failed node returns, it can be updated again.

Example

A global application may have database nodes in India, USA, and Europe.

- A user from India writes data to the India node.
- A user from Europe writes data to the Europe node.
- Later, the nodes synchronize their copies.

This improves availability and reduces access delay.

Advantages

- No single master
- No single point of failure
- Better write scalability
- High availability
- Any node can accept read/write requests
- Suitable for distributed systems such as Cassandra-style databases

Limitations

- Write conflicts may occur
- Consistency is harder to maintain
- Nodes must coordinate updates
- Conflict resolution is required
- Network communication overhead increases

Note: The biggest complication in peer-to-peer replication is **consistency**. Since writes can happen at different places, two users may update the same record at the same time, causing a write-write conflict.

Sharding and Replication

Large NoSQL systems often combine both **sharding** and **replication**.

- **Sharding** gives scalability.
- **Replication** gives availability and fault tolerance.

In simple words:

- Sharding divides data across nodes, while replication keeps copies of each shard on multiple nodes.

- With master-slave replication and sharding, there may be multiple masters, but each data item has only one master.
- With peer-to-peer replication and sharding, each shard may be copied to multiple nodes.

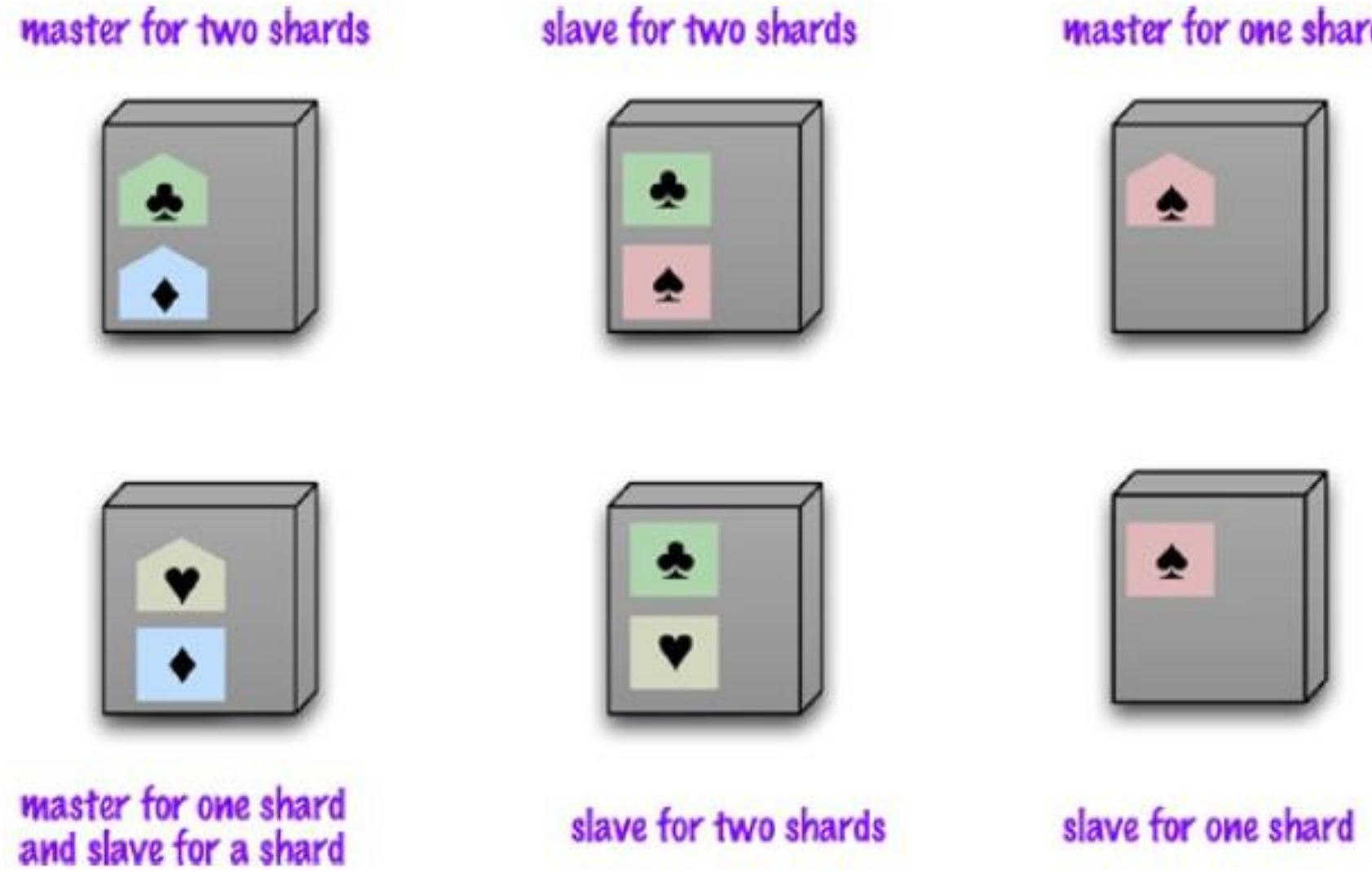


Figure 2.4. Using master-slave replication together with sharding

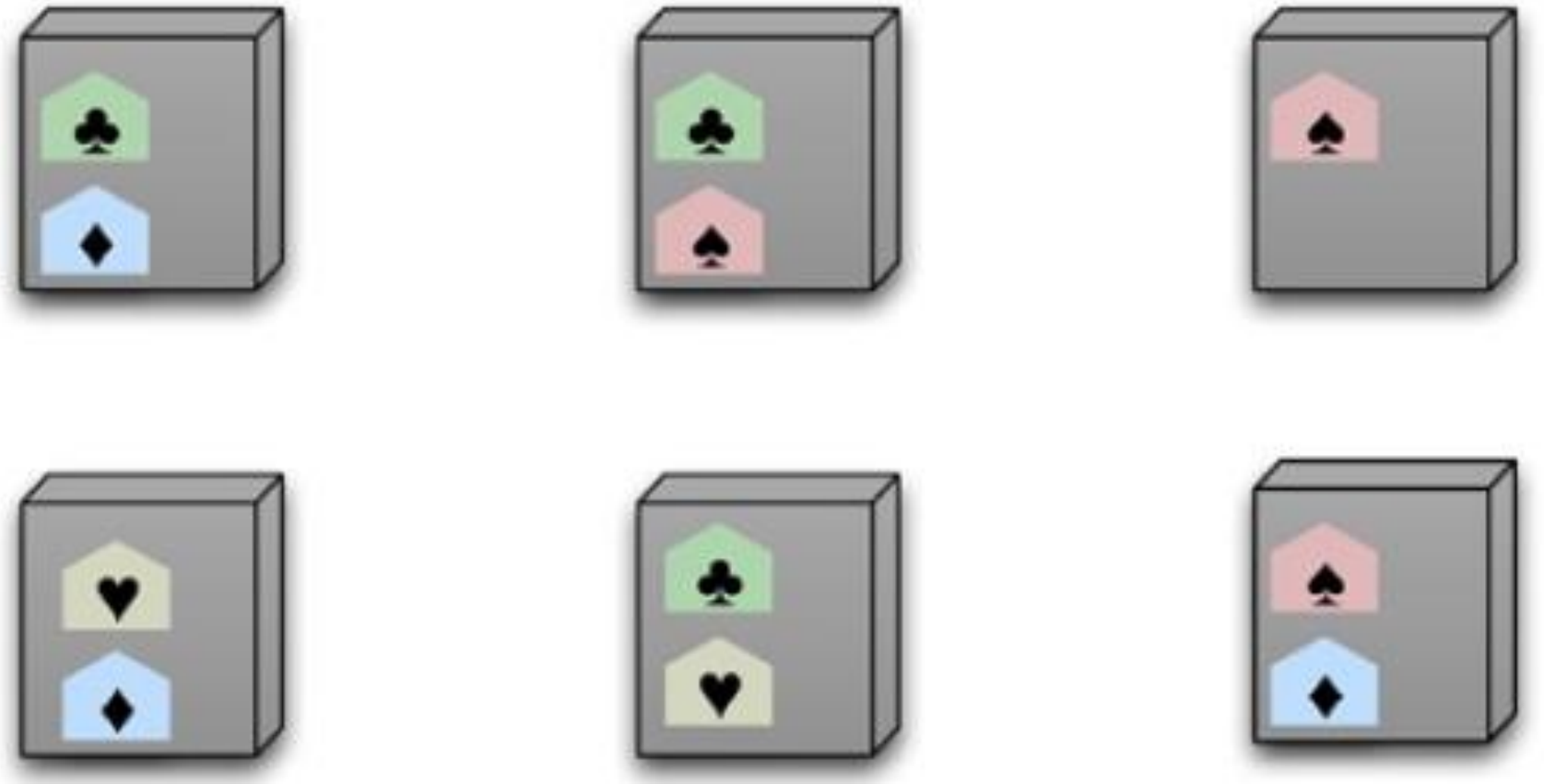


Figure 2.5. Using peer-to-peer replication together with sharding

There are two styles of distributing data:

- Sharding distributes different data across multiple servers, so each server acts as the single source for a subset of data.
- Replication copies data across multiple servers, so each bit of data can be found in multiple places.

A system may use either or both techniques.

Replication comes in two forms:

- Master-slave replication makes one node the authoritative copy that handles writes while slaves synchronize with the master and may handle reads.
- Peer-to-peer replication allows writes to any node; the nodes coordinate to synchronize their copies of the data.

Master-slave replication reduces the chance of update conflicts but peer-to-peer replication avoids loading all writes onto a single point of failure.

Comparison Table

Feature	Sharding	Master-Slave Replication	Peer-to-Peer Replication
Main purpose	Scalability	Read scalability and backup	Availability and write scalability
Data placement	Different data on different nodes	Same data copied from master to slaves	Same data copied across equal nodes
Write handling	Each shard handles its data	Master handles writes	Any node can handle writes
Read handling	From relevant shard	Master or slaves	Any node
Failure handling	Depends on shard availability	Slaves can support reads; master failure affects writes	Any node failure can be tolerated
Main problem	Shard key and rebalancing	Master bottleneck	Write conflicts and consistency

Consistency

Consistency means that data should remain correct, meaningful, and predictable after read and write operations.

In simple words:

Consistency means every user should see correct data, and the database should not give conflicting or incomplete results.

- In a single relational database, consistency is usually maintained using **transactions**.
- But in NoSQL databases, especially distributed databases, data may be stored on many nodes. Therefore, maintaining consistency becomes more difficult.

Why consistency is difficult in NoSQL?

- In a distributed NoSQL database, data may be copied across multiple nodes. When one node is updated, other nodes may not receive the update immediately. During this time, different users may see different values.

Example:

Node 1 → Updated value

Node 2 → Old value

Node 3 → Old value

This creates temporary inconsistency.

Update Consistency: Update consistency deals with correctness when two or more users try to update the same data at the same time. This situation is called a **write-write conflict**.

Example from textbook

Suppose two users, Martin and Pramod, both notice that a company phone number is wrong. Both try to update the same phone number at the same time, but each uses a slightly different format. This creates a **write-write conflict** because two people are updating the same data item at the same time.

Lost update problem:

If the database accepts Martin's update first and then Pramod's update, Martin's update may be overwritten. This is called a **lost update**.

Original Phone Number

↓

Martin updates

↓

Pramod updates

↓

Martin's update is lost

It is a failure of consistency because the second update is based on an old state of the data.

Methods to handle update conflicts

1. Pessimistic approach

In the **pessimistic approach**, the system prevents conflicts before they happen. The common method is **locking**.

Example:

User 1 gets lock → updates data

User 2 waits → then reads latest data → then updates if needed

Only one user can update the data at a time.

2. Optimistic approach

- In the **optimistic approach**, the system allows users to try updates, but checks whether the data has changed before saving.

Example:

Martin reads value

Pramod reads same value

Martin updates successfully

Pramod tries to update

System detects old value and rejects Pramod's update

- Then Pramod must read the latest value and decide whether to update again.

Sequential consistency

- In distributed systems, different nodes may apply updates in different orders. **Sequential consistency** means all nodes apply operations in the same order.

Example:

Correct order: Update A → Update B

Problem:

Node 1 applies A then B

Node 2 applies B then A

This can lead to different values on different nodes. This problem is important in peer-to-peer replication.

Read Consistency

- **Read consistency** deals with correctness when users read data while another update is happening. This situation is called a **read-write conflict**.

Example: Order and shipping charge

Suppose an order contains line items and a shipping charge. When a new line item is added, the shipping charge must also be recalculated.

Problem:

Step 1: Martin adds a new line item

Step 2: Pramod reads line items and shipping charge

Step 3: Martin updates shipping charge

Pramod reads the order in the middle of Martin's update. He sees new line items but old shipping charge. This is an **inconsistent read** or **read-write conflict**.

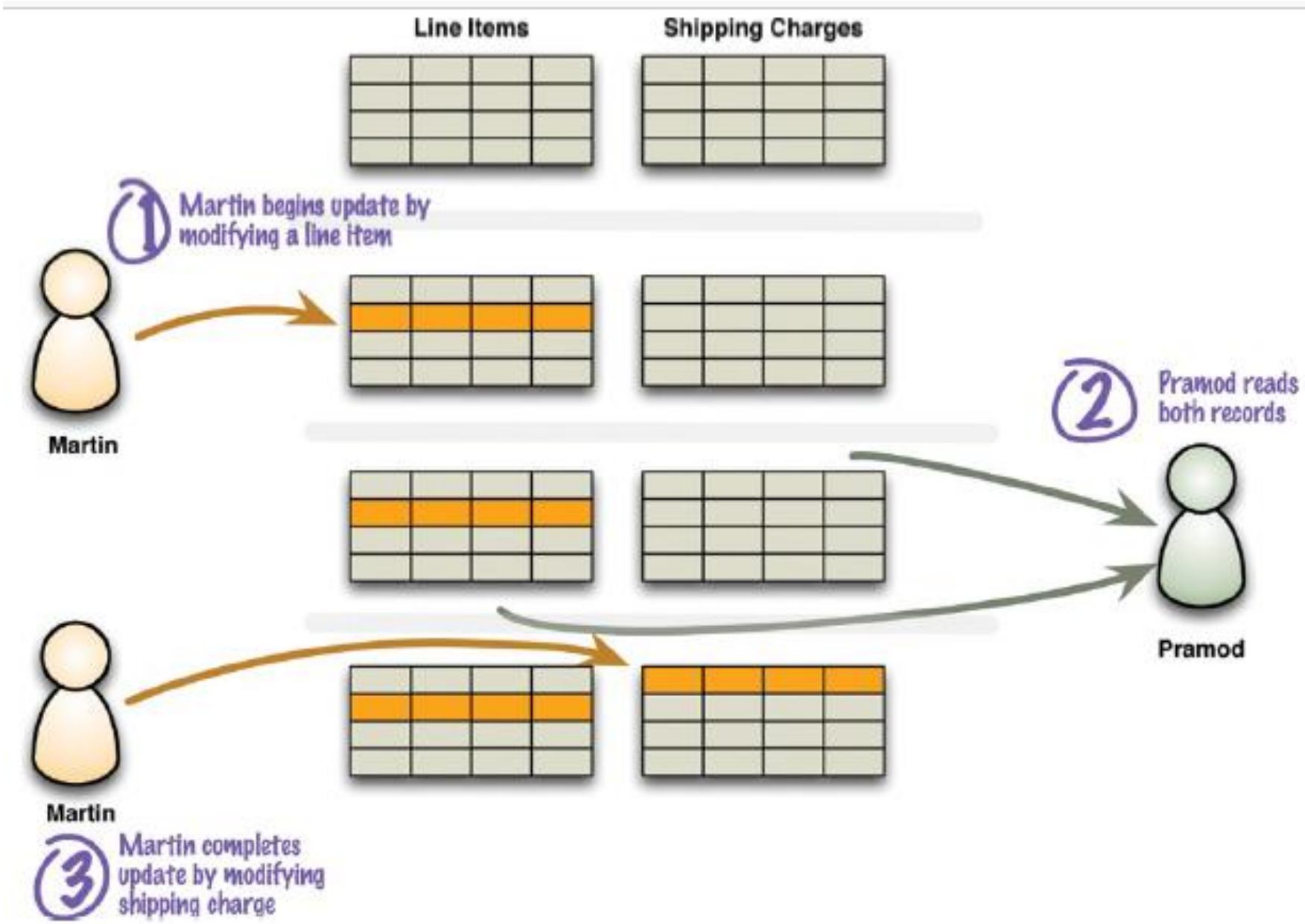


Figure 2.6. A read-write conflict in logical consistency

Logical consistency

Logical consistency means different related data items should make sense together.

Example:

Order items total = ₹5000

Shipping charge = calculated for old order amount

Here, the data exists, but it does not logically match.

- Relational databases avoid this by using transactions. If Martin wraps both updates in one transaction, Pramod will either read the data before the update or after the update, but not halfway.

Inconsistency Window:

The **inconsistency window** is the time period during which inconsistent data may be visible.

Example:

Update starts → Some data is old and some is new → Update completes

- The time between update start and update completion is the inconsistency window.
- In NoSQL systems, if an update affects multiple aggregates, there may be a short time where users can read inconsistent data.

Replication Consistency:

- **Replication consistency** means the same data item should have the same value when read from different replicas.
- In a distributed database, the same data may be copied to multiple nodes. If one node is updated before the others, different users may read different values.

Example: Hotel room booking

Suppose there is one last hotel room. The booking system runs on many nodes.

Mumbai node → room booked

Boston node → update received

London node → update not yet received

One user may see the room as booked, while another user may still see it as available. This is **replication inconsistency**.

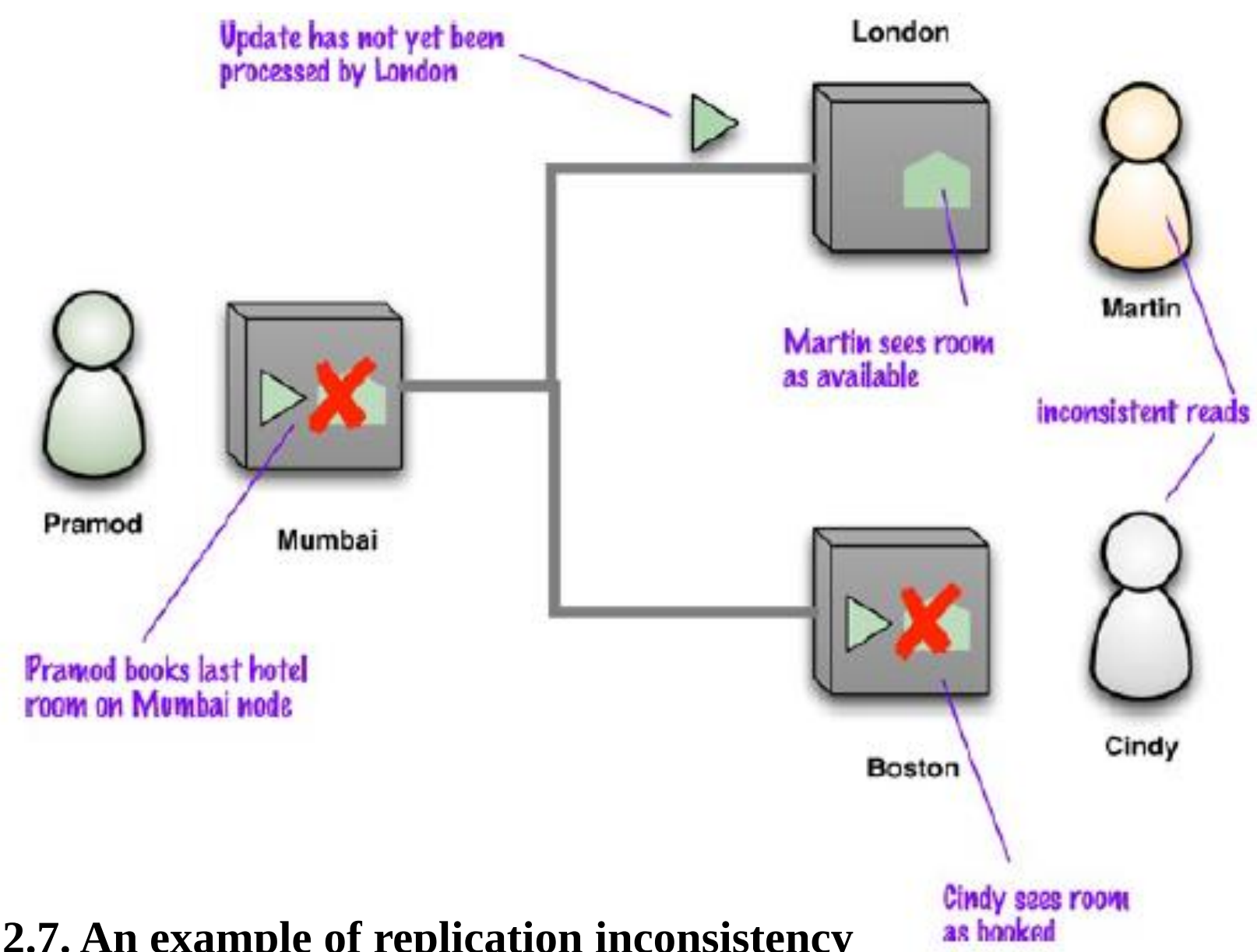


Figure 2.7. An example of replication inconsistency

Eventual Consistency:

Eventual consistency means that all replicas may not be immediately consistent, but if no new updates occur, all replicas will eventually reach the same value.

In simple words:

Eventual consistency means data may be temporarily different on different nodes, but after some time all nodes become consistent.

Example:

Time 1:

Node 1 → Booked

Node 2 → Available

Node 3 → Available

Time 2:

Node 1 → Booked

Node 2 → Booked

Node 3 → Booked

- This situation is called eventually consistent because updates will fully propagate after some time. Data that is out of date is called **stale data**.

Relaxing Consistency

- **Relaxing consistency** means deliberately allowing some temporary inconsistency to improve performance, availability, or response time.

In simple words:

- Relaxing consistency means we accept slightly stale or temporarily inconsistent data when strict consistency is too costly.
- Consistency is desirable, but sometimes avoiding all inconsistency requires unacceptable sacrifices in other system qualities. Different domains have different tolerance levels for inconsistency.

Why relax consistency?

Consistency may be relaxed to achieve:

- Better performance
- Faster response time
- Higher availability
- Better scalability
- Continued service during network failure

Even relational databases sometimes relax consistency through lower isolation levels, such as read-committed, to improve performance. Some large systems also avoid transactions for performance reasons, especially when sharding is used.

CAP Theorem:

The **CAP theorem** is commonly used to explain why distributed systems may need to relax consistency.

CAP stands for:

Term	Meaning
Consistency	All users see correct/latest data
Availability	A non-failing node responds to requests
Partition tolerance	System continues despite network breakages

The basic statement of CAP is that, among **Consistency, Availability, and Partition tolerance**, a distributed system can only fully guarantee two at the same time.

Main idea of CAP in NoSQL

- In real distributed systems, network partitions can happen. So systems must be partition tolerant. When a partition occurs, the system must trade off between:

  Consistency ↔ Availability

- This is not always a binary decision. A system may trade off a little consistency to gain some availability, depending on application needs.

Example of Relaxing Consistency

Hotel booking example

- If two nodes cannot communicate and both keep accepting reservations, two people may book the last room.
- For some hotel systems, this may be unacceptable. But for some travel businesses, a small amount of overbooking may be acceptable because cancellations and no-shows happen.
- So, whether consistency can be relaxed depends on the business domain.

Shopping cart example

- A system may allow users to keep writing to their shopping cart even during network failures. If multiple cart versions are created, the checkout process can merge the items into one cart.
- This is acceptable because the user can still review the cart before final purchase.

Version Stamps

- In NoSQL databases, especially distributed databases, many users or nodes may try to update the same data. This can create **concurrency conflicts**. Here, transactions are useful for consistency, but even transactional systems have limitations when user actions take a long time. Version stamps help manage such situations.

Simple definition

A version stamp is a value stored with a record that changes whenever the record is updated.

In simple words:

A version stamp helps the system know whether the data has changed after it was last read.

Why Version Stamps Are Needed

- When a user reads data and later updates it, the data may have been changed by another user in between.

Example

- Suppose a customer places an order. During the process, the customer views product price, enters payment details, and confirms the order. This full activity is a **business transaction**. But the database cannot keep locks open during the entire user activity because the user may take time to complete the process.

So, before saving the final update, the system should check:

Has the data changed since the user first read it?

Version stamps help answer this question.

Suppose two teachers are updating the same student marks.

Initial record:

Marks = 80, Version = 1

Teacher A reads:

Marks = 80, Version = 1

Teacher B also reads:

Marks = 80, Version = 1

Teacher A updates marks:

Marks = 85, Version = 2

Teacher B tries to update marks using old version:

Marks = 90, Expected Version = 1

System checks:

Current Version = 2

Expected Version = 1

So, Teacher B's update is rejected or sent for conflict handling.

Types of Version Stamps

Several ways to create version stamps:

1. Counter-Based Version Stamps
2. GUID-Based Version Stamps
3. Content hash Version Stamps
4. Timestamp-Based Version Stamps
5. Composite Version Stamps

Uses of Version Stamps

Version stamps are used for:

Use	Explanation
Conflict detection	Detects if data changed between read and write
Optimistic locking	Allows update only if version is unchanged
Stale data prevention	Avoids updates based on old data
Session consistency	Helps users see their own updates
Distributed consistency	Helps detect conflicts between multiple nodes
Peer-to-peer replication	Vector stamps identify conflicting updates

Working with Cassandra

- Refer the notes

Thank You